

Distributed & Operating Systems

Bazar.com [Lab1 & Lab2]

Dr. Samer Arandi

Yousef Yaser Hajeer 12217097

Hazem Simreen Mahamdah 12217328

1. Overview

Lab 1: Bazar.com represents a light-weight two-tier web book store created using microservices and REST HTTP calls. This system is loaded with four books in total on two subjects (distributed systems and undergraduate school). These four books are represented by three different services: a front end, catalog, and order server, all of which run separately. The application is coded in JavaScript (Node.js/Express).

Lab 2: Lab 2 extends the system to handle higher load by introducing replication, in-memory caching, and cache consistency. Three new books are added to the catalog. Both the catalog and order servers are each replicated across two instances, and the front-end gains an in-memory cache for read queries along with a load balancer to distribute requests across replicas.

2. System Architecture

2.1 Lab 1: Three-Service Design

The Lab 1 system consists of three microservices:

- ❖ Front-End Service: Single entry point for all client requests. Exposes search, info, and purchase endpoints, and forwards them to the appropriate backend service.
- ❖ Catalog Service: Manages all book metadata (ID, title, topic, price, quantity) stored in a CSV file. Supports query-by-topic, query-by-item, and update operations.

Order Service: Handles purchase requests. Verifies stock with the Catalog Service, decrements quantity on success, and logs orders to a text file.

2.2 Lab 2: Five-Service Replicated Design

Lab 2 expands the system to five microservices:

- ❖ Front-End Service (1 instance): Unchanged role, but now also manages an in-memory cache and round-robin load balances across catalog and order replicas.
- ❖ CatalogReplica1 & CatalogReplica2: Two identical catalog service instances, each with its own catalog.csv, running on separate ports.
- ❖ OrderReplica1 & OrderReplica2: Two identical order service instances, each with its own orders.txt log.

3. How It Works

3.1 Search

- ❖ Lab 1: A client sends GET /search/:topic to the front-end, which relays it to the Catalog Service. The catalog reads its CSV, filters by topic, and returns a JSON array of matching titles and IDs.
- ❖ Lab 2: The front-end first checks the in-memory cache. On a miss, it round-robins to one of the two Catalog Replicas, fetches the result, stores it in the cache, and returns it to the client.

3.2 Info

- ❖ Lab 1: A client sends GET /info/:id to the front-end. The Catalog Service looks up the book by ID and returns title, quantity, and price as JSON.
- ❖ Lab 2: Same flow, but the front-end checks the cache first. Cache hits are returned immediately without any backend call, significantly reducing latency for repeated lookups.

3.3 Purchase

- ❖ Lab 1: A client sends POST /purchase/:id. The front-end forwards it to the Order Service, which verifies stock with the Catalog Service, decrements quantity via PUT /update/:id, and logs the order to orders.txt.
- ❖ Lab 2: The front-end round-robins to an Order Replica. That replica verifies stock, sends a cache invalidation request to the front-end (/cache/:id) before writing, then calls PUT /update/:id on BOTH Catalog Replicas to keep them in sync. The order is logged and a response is returned.

3.4 Data Persistence

- ❖ Lab 1: The Catalog Service writes to `catalog.csv` and the Order Service appends to `orders.txt`. Both files survive service restarts.
- ❖ Lab 2: Each replica maintains its own copy of these files. Writes are propagated to both replicas so their data files stay consistent with one another.

3.5 Concurrency

Both labs rely on Node.js's non-blocking I/O model and Express's built-in concurrent request handling. File writes are kept synchronous to prevent partial writes. In Lab 2, the two replicas per service further distribute concurrent load, reducing contention on any single instance.

4. Replication and Load Balancing

The front-end uses round-robin load balancing, maintaining two counters: one for catalog replicas and one for order replicas-that alternate with each incoming request. This distributes load evenly without requiring knowledge of replica state or health.

To keep replicas consistent, any write operation (purchase or stock update) is applied to both catalog replicas, not just the one selected by the load balancer. This dual-write strategy ensures both replicas always reflect the same data after every modification.

5. Caching and Cache Consistency

The in-memory cache is implemented as a JavaScript Map inside the front-end process, keyed by book ID. Storing it in-process eliminates REST overhead for every cache lookup. Read requests check the cache before hitting any backend replica; write requests always bypass the cache entirely.

Cache consistency is maintained via server-push invalidation. Before a backend replica commits any write, it sends a `DELETE /cache/:id` request to the front-end, which immediately evicts that book's entry. The next read for that book fetches fresh data from a catalog replica and re-populates the cache, ensuring no stale data is ever served after a write.

6. Performance Measurements

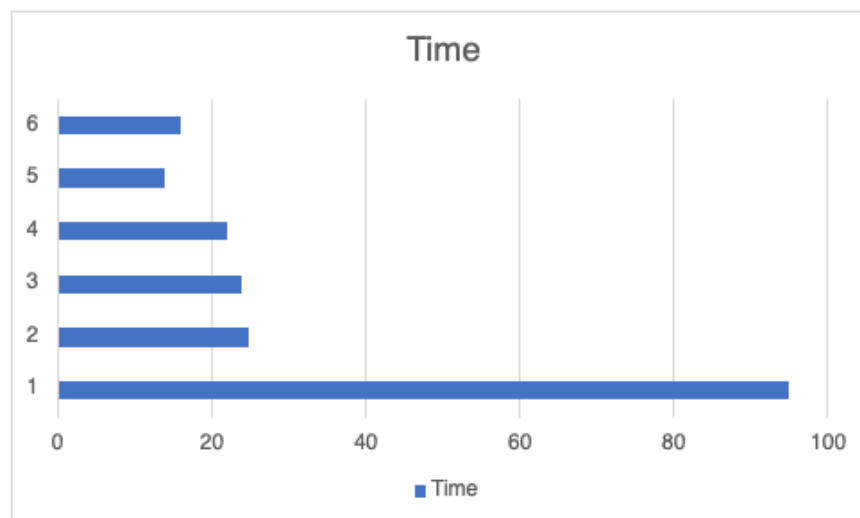
6.1 Response Time: With vs. Without Cache

Request	Type	Response Time (ms)
1	MISS	95 ms
2	HIT	25 ms
3	HIT	24 ms
4	HIT	22 ms
5	HIT	14 ms
6	HIT	16 ms
Average HIT Time		20.2 ms
Cache Improvement		78.7% faster

Analysis

The results show that caching significantly improved system performance. The first request experienced higher latency because the frontend service had to retrieve the data from the backend catalog service and read the CSV database. After the data was cached in memory, subsequent requests were much faster.

The average cache hit response time was 20.2 ms compared to 95 ms for the cache miss, resulting in an approximate performance improvement of 78.7%.



6.2 Cache Invalidation Overhead:

Operation	Cache State	Response Time
POST /purchase/2	Cache Invalidated	230ms
GET /info/2	MISS	60ms
GET /info/2	Hit	22ms

Analysis

During the purchase operation, the frontend service invalidated the cached book data to maintain consistency with the updated catalog stock. This introduced additional overhead because the system needed to update the catalog service and remove stale cache entries.

After cache invalidation, the next request became a cache miss again and required fetching fresh data from the backend service. As a result, the response time increased compared to normal cache hit operations.

This experiment demonstrates that the implemented cache consistency mechanism successfully prevents stale data from being returned to users after catalog updates or purchases.



7. How to run

7.1 Prerequisites

- Docker & Docker Compose installed on the host machine
(Docker Desktop on macOS/Windows, or Docker Engine + Compose plugin on Linux).
- Node.js v18+
(only required if running services manually outside Docker).

7.2 Lab 1: Running with Docker

From the repository root, run the following commands:

- ❖ Clone the repo:
`git clone https://github.com/yousefh2004/BazarYH.git`
`cd BazarYH`
- ❖ Build and start:
`docker compose up --build`
- ❖ Test search:
`curl http://localhost:5050/search/distributed%20systems`
- ❖ Test info:
`curl http://localhost:5050/info/1`
- ❖ Test purchase:
`curl -X POST http://localhost:5050/purchase/1`
- ❖ Stop services:
`docker compose down`

7.3 Lab 1: Running Manually (without Docker)

Open three separate terminals from the repository root:

- ❖ Terminal 1: Catalog Service
cd CatalogReplica1
npm install
node index.js
- ❖ Terminal 2: Order Service
cd OrderReplica1
npm install
node index.js
- ❖ Terminal 3: Front-End Service
cd FrontendService
npm install
node index.js
- ❖ The front-end will be accessible at:
http://localhost:5050

Make sure the host/port environment variables in each service configuration point to localhost with the correct ports before running manually.

7.4 Lab 2: Running with Docker

The compose. yaml at the repository root already defines all five Lab 2 containers.

From the repository root:

- ❖ Build and start all five services:
docker compose up --build

```
Running 12/11
✓ bazaryh-catalog-service-1      Built      0.05s
✓ bazaryh-catalog-service-2      Built      0.05s
✓ bazaryh-order-service-1        Built      0.05s
✓ bazaryh-order-service-2        Built      0.05s
✓ bazaryh-frontend-service       Built      0.05s
✓ Network bazaryh_default        Created    0.05s
✓ Container bazaryh-catalog-service-2-1 Created    0.35s
✓ Container bazaryh-catalog-service-1-1 Created    0.35s
✓ Container bazaryh-order-service-2-1 Created    0.35s
✓ Container bazaryh-order-service-1-1 Created    0.35s
✓ Container bazaryh-frontend-service-1 Created    0.15s
Attaching to catalog-service-1-1, catalog-service-2-1, frontend-service-1, order-service-1-1, order-service-2-1
order-service-1-1 | Order Replica 1 running on 5002
catalog-service-2-1 | Catalog Replica 2 running on 5001
catalog-service-1-1 | Catalog Replica 1 running on 5001
order-service-2-1 | Order Replica 2 running on 5002
frontend-service-1 | Frontend service running on port 5000
View in Docker Desktop View Config Enable Watch
```

- ❖ Verify replicas are running:
docker compose ps

Containers [Give feedback](#)

Container CPU usage 0.00% / 800% (8 CPUs available) Container memory usage 190MB / 3.74GB [Show charts](#)

Search ☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Actions
☐	sqlserver	96758d45d752	mssql/server	1433:1433	0%	0B / 0B	0%	
☒	bazaryh	-	-	-	0%	190MB / 19.14GE	4.85%	
☐	catalog-service	e2eef5242c64	bazaryh-ca	5001:5001	0%	40.93MB / 3.83Gi	1.04%	
☐	catalog-service	6d4d278f6f99	bazaryh-ca	5003:5001	0%	38.78MB / 3.83Gi	0.99%	
☐	order-service	0b45afa5c330	bazaryh-ors	5002:5002	0%	34.97MB / 3.83Gi	0.89%	
☐	order-service	15e4bc83d012	bazaryh-ors	5004:5002	0%	37.09MB / 3.83Gi	0.95%	
☐	frontend-serv	017676cb41ad	bazaryh-fro	5050:5000	0%	38.23MB / 3.83Gi	0.98%	

- ❖ Test with caching (repeat same request twice):

curl <http://localhost:5050/info/1>

```
[mazen@192 ~ % curl http://localhost:5050/info/1]
[{"id":1,"title":"How to get a good grade in DOS in 40 minutes a day","topic":"distributed systems","stock":0,"price":50}]
mazen@192 ~ % curl http://localhost:5050/info/1
{"id":1,"title":"How to get a good grade in DOS in 40 minutes a day","topic":"distributed systems","stock":0,"price":50}
mazen@192 ~ %
```

- ❖ Run it twice: the second call should be a cache hit.

```
frontend-service-1 | Frontend service running on port 5000
frontend-service-1 | CACHE MISS
catalog-service-1-1 | [CATALOG 1] info(1)
frontend-service-1 | CACHE HIT
```

- ❖ Trigger cache invalidation:

curl -X POST <http://localhost:5050/purchase/1>

```
[mazen@192 ~ % curl -X POST http://localhost:5050/purchase/1]
[{"msg":"purchase successful","title":"How to get a good grade in DOS in 40 minutes a day"}]
mazen@192 ~ % curl http://localhost:5050/info/1
{"id":1,"title":"How to get a good grade in DOS in 40 minutes a day","topic":"distributed systems","stock":1,"price":50}
mazen@192 ~ %
```


- ❖ Then re-run:
curl http://localhost:5050/info/1
to observe a cache miss.

```
catalog-service-1-1 | [CATALOG 1] info(1)
catalog-service-1-1 | [CATALOG 1] update(1) new stock=1
catalog-service-2-1 | [CATALOG 2] replicated update(1) stock=1
frontend-service-1 | CACHE MISS
```

- ❖ Stop all services:
docker compose down

```
mazen@192 BazarYH % docker compose down
docker compose up --build
[+] Running 6/6
  ✓ Container bazaryh-frontend-service-1    Removed
  ✓ Container bazaryh-order-service-2-1     Removed
  ✓ Container bazaryh-order-service-1-1     Removed
  ✓ Container bazaryh-catalog-service-2-1   Removed
  ✓ Container bazaryh-catalog-service-1-1   Removed
  ✓ Network bazaryh_default                 Removed
[+] Building 4.7s (44/44) FINISHED
```

7.5 Lab 2: Running Manually (without Docker)

Open five separate terminals from the repository root:

- ❖ Terminal 1
cd CatalogReplica1
npm install
node index.js
- ❖ Terminal 2
cd CatalogReplica2
npm install
node index.js
- ❖ Terminal 3
cd OrderReplica1
npm install
node index.js
- ❖ Terminal 4
cd OrderReplica2
npm install
node index.js

- ❖ Terminal 5

```
cd FrontendService
npm install
node index.js
```

Ensure each service environment configuration (host/port for replicas it connects to) is set to localhost with the correct port numbers before starting.

7.6 Expected Output

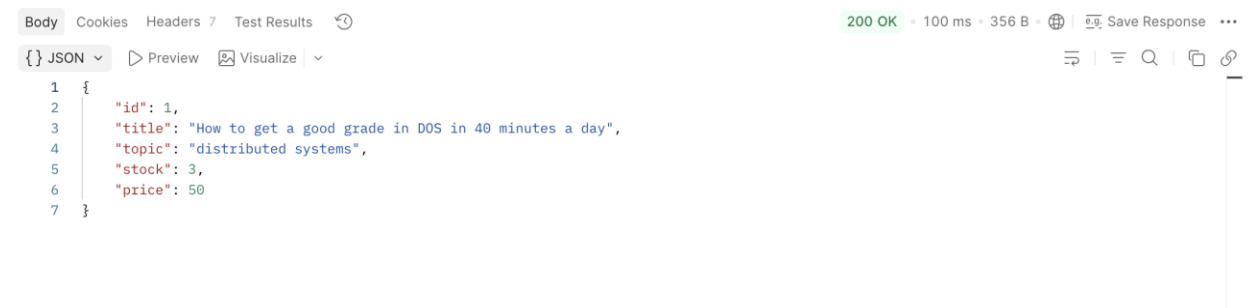
- ❖ A successful search returns a JSON array of books matching the topic.
(Get <http://localhost:5050/search/distributed systems>)



The screenshot shows a web browser's developer console with the 'Body' tab selected. The response is a JSON array of two book objects. The status bar at the top indicates a 200 OK response with a 130 ms latency and 341 B of data. The JSON content is as follows:

```
1 [
2   {
3     "id": 1,
4     "title": "How to get a good grade in DOS in 40 minutes a day"
5   },
6   {
7     "id": 2,
8     "title": "RPCs for Noobs"
9   }
10 ]
```

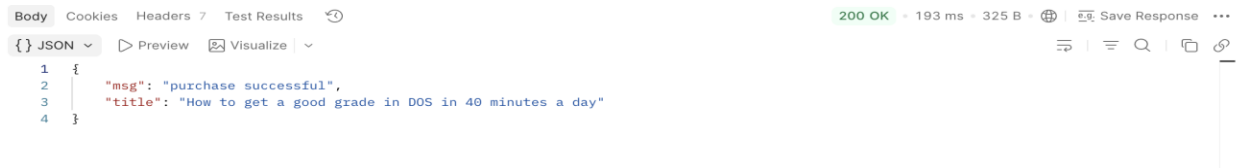
- ❖ A successful info request returns a JSON object with:
title
price
Quantity
(Get <http://localhost:5050/info/1>)



The screenshot shows a web browser's developer console with the 'Body' tab selected. The response is a JSON object containing details for a specific book. The status bar at the top indicates a 200 OK response with a 100 ms latency and 356 B of data. The JSON content is as follows:

```
1 {
2   "id": 1,
3   "title": "How to get a good grade in DOS in 40 minutes a day",
4   "topic": "distributed systems",
5   "stock": 3,
6   "price": 50
7 }
```

- ❖ A successful purchase prints:
bought book <book_name>
in the server log and returns a success JSON response to the client.
(Post <http://localhost:5050/purchase/1>)



- ❖ A purchase on an out-of-stock book returns a JSON error response with an appropriate message.
(Post <http://localhost:5050/purchase/1>)



8. Design Tradeoffs

- ❖ CSV vs. SQLite: CSV was chosen for simplicity and zero setup. The tradeoff is reduced safety around concurrent writes. SQLite would offer atomic transactions at minimal operational cost.
- ❖ Integrated cache vs. separate cache service (Lab 2): Embedding the cache in the front-end avoids a REST hop per lookup. A separate service would be more modular but slower.
- ❖ Server-push invalidation vs. TTL (Lab 2): Invalidation on write provides strong consistency. TTL is simpler but risks serving stale data during the TTL window.
- ❖ Round-robin load balancing (Lab 2): Simple and stateless, but does not account for replica health. A downed replica still receives traffic without additional health checks.
- ❖ Dual-write consistency (Lab 2): Updating both catalog replicas on every purchase ensures consistency but adds write latency. A leader-follower model would scale better.

9. Possible Improvements and Extensions

- ❖ SQLite upgrade: Replacing CSV with SQLite would provide atomic transactions and safer concurrent reads/writes across all replicas.
- ❖ LRU cache eviction (Lab 2): A bounded cache with LRU replacement would prevent unbounded memory growth and keep frequently accessed books prioritized.
- ❖ Replica health checks (Lab 2): The load balancer could probe replica liveness and remove unresponsive nodes from rotation.
- ❖ Leader-follower replication (Lab 2): A primary-secondary model would reduce write complexity and allow the secondary to serve as a hot standby.
- ❖ Front-end replication: The front-end is a single point of failure in both labs. Replicating it behind a hardware or software load balancer would address this.
- ❖ Stock restocking simulation: A background process that periodically increments stock levels would simulate real-world inventory replenishment as described in the spec.

10. Known Limitations

- ❖ Under high concurrency, simultaneous purchases of the same book can result in a race condition when reading and writing the CSV stock count. This is acceptable at lab scale but would need a locking mechanism in production.
- ❖ If a backend service or replica is unreachable, the front-end returns an error rather than a graceful fallback, as no retry or circuit-breaking logic is implemented.
- ❖ In Lab 2, if the front-end crashes, the cache is lost entirely on restart. There is no cache warming mechanism, so the first wave of requests after a restart will all be cache misses.